Reference Notes

Jonathan Ma

August 11, 2026

Contents

I Literature Review **3**

1 Summary of Related Work **3**

2 Extended Article Notes **4**
- 2.1 Lets Verify Step by Step . . . 4
- 2.2 Towards Robust PRMs via Noise-Aware Learning . . . 5
- 2.3 CoLD: Counterfactually Guided Length Debiasing for PRMs in Math Reasoning . . . 7
- 2.4 PathFinder-PRM: Error-Aware Hierarchical Supervision . . . 8
- 2.5 Stop Summation: Min-Form Credit Assignment Is All Process Reward Model Needs for Reasoning . . . 9
- 2.6 Adversarial Training for Process Reward Models . . . 11
- 2.7 Preventing Process Reward Model Hacking When Training Large Language Models on Verifiable Rewards . . . 12
- 2.8 Reward Under Attack: Analyzing the Robustness and Hackability of PRMs . . . 13

3 Other Related Work **15**
- 3.1 Towards Hierarchical Multi-Step Reward Models for Enhanced Reasoning in LLMs . . . 15
- 3.2 SCAN: Self-Denoising MC Annotation for Robust Process Reward Learning . . . 17
- 3.3 Know What You Don't Know: Uncertainty Calibration of PRMs . . . 18
- 3.4 SOCRATIC-PRMBENCH: Benchmarking Process Reward Models with Systematic Reasoning Patterns . . . 19
- 3.5 PRISM: Probabilistic Reward Model with Inherent Structural Modeling . . . 21
- 3.6 Teach a Reward Model to Correct Itself: Reward Guided Adversarial Failure Discovery for Robust Reward Modeling . . . 22

II Notes **24**

4 Reinforcement Learning Overview **24**

5 Markov Decision Processes (MDPs), Returns, and Policies **24**

6 Expected Return **25**
6.1 Optimal Policy 25
6.2 Bellman Equations 26

7 Tabular Reinforcement Learning **26**
7.1 Dynamic Programming 26

Part I
Literature Review

1. Summary of Related Work

Process Reward Models (PRMs) were introduced by *Let's Verify Step by Step*, which demonstrated that dense, step-level supervision substantially improves reasoning compared to outcome-only rewards. The paper introduced the PRM800K dataset and established process supervision as the standard paradigm for reasoning reward models. However, PRMs were studied primarily as inference-time verifiers for best-of-N search rather than reward functions within reinforcement learning, leaving questions regarding robustness under optimization largely unexplored.

Recent work has shown that PRMs suffer from multiple forms of reward misspecification. *CoLD* identifies a causal shortcut in which reward models systematically favor verbose reasoning over concise but equivalent solutions. By modeling reasoning length as a confounding variable, the authors introduce counterfactual debiasing to separate semantic correctness from verbosity. In contrast, *Towards Robust PRMs via Noise-Aware Learning* argues that the dominant source of error lies in the supervision itself: Monte Carlo annotations depend on the rollout policy rather than the intrinsic correctness of a reasoning step, producing systematic false positives and false negatives. The proposed reflection-aware relabeling and iterative denoising framework explicitly treats noisy supervision as a robustness problem.

Another line of work improves the reward model through more informative supervision. *PathFinder-PRM* decomposes reward prediction into intermediate error identification followed by reward estimation, demonstrating that distinguishing mathematical and logical inconsistencies leads to more reliable reward prediction. *Adversarial Training for Process Reward Models* instead formulates reward model training as a two-player game in which a generator continually produces increasingly difficult reasoning errors. Rather than relying on a fixed dataset, the reward model adapts to an evolving distribution of adversarial reasoning trajectories, improving generalization to out-of-distribution reasoning errors.

While these methods improve PRM robustness, they largely focus on improving reward prediction rather than studying optimization-induced failure. *Reward Under Attack* explicitly demonstrates that current PRMs are highly vulnerable to optimization: reinforcement learning policies can achieve nearly perfect PRM rewards while maintaining poor reasoning accuracy, revealing that PRMs often reward fluent reasoning rather than logically correct reasoning. Rather than proposing a new reward model, the paper establishes a comprehensive evaluation framework based on adversarial perturbations, gradient-based attacks, and reinforcement learning, providing one of the strongest empirical demonstrations of PRM reward hacking to date.

Only a limited number of papers directly address reward hacking during optimization. *Stop Summation* argues that reward hacking arises from the reinforcement learning objective itself, replacing cumulative process rewards with a minimum-future-reward objective that reduces pathological credit assignment. Similarly, *Preventing Process Reward Model Hacking When Training Large Language Models on Verifiable Rewards* views PRMs through the lens of reward shaping, adapting policy-invariant shaping methods to provide dense intermediate rewards while preserving the optimal policy under verifiable rewards. These approaches suggest that robustness depends not only on the

reward model, but also on the optimization algorithm consuming those rewards.

Overall, the current literature approaches robust PRMs from three complementary perspectives: improving the reward signal through debiasing or better supervision, improving the reward model through adversarial or hierarchical training, and modifying the reinforcement learning objective to reduce reward exploitation. Despite these advances, there remains relatively little work that jointly considers robust reward modeling and robust optimization, particularly under distribution shift and adversarial policy optimization. This intersection appears particularly relevant from the perspective of data-driven optimization and reinforcement learning.

2. Extended Article Notes

2.1. Lets Verify Step by Step

Link

TLDR: Foundational PRM/process-supervision paper for background. Uses human step-level labels and compares process vs outcome supervision. Empirical study and releases a dataset with 800k labels for training

Problem. Large language models have become increasingly capable of multi-step reasoning through chain-of-thought generation, yet they still frequently make subtle logical errors that invalidate otherwise convincing solutions. Existing reward models are typically trained using *outcome supervision*, which labels only the final answer as correct or incorrect. This provides sparse feedback and requires the reward model to infer where an error occurred, creating a difficult credit assignment problem. The authors investigate whether *process supervision*, which provides human feedback on each intermediate reasoning step, can produce more reliable reward models despite its higher annotation cost.

Approach. The authors compare outcome-supervised reward models (ORMs) and process-supervised reward models (PRMs) on mathematical reasoning tasks using GPT-4-based models. They collect **PRM800K**, a dataset containing approximately 800,000 human-annotated step-level labels across 75,000 reasoning traces, where annotators identify the first incorrect reasoning step rather than only the final outcome. PRMs are trained to predict the correctness of every reasoning step, and complete solutions are scored by aggregating these step-level probabilities. The authors additionally introduce an active learning strategy that prioritizes labeling convincing but incorrect solutions, increasing annotation efficiency by approximately 2.6×. Experiments on the MATH benchmark demonstrate that process supervision consistently outperforms outcome supervision, achieving 78.2% accuracy under best-of-1860 search compared to 72.4% for ORMs while also generalizing better to unseen STEM examination problems.

Limitations. The approach relies heavily on expensive human annotation, making PRM800K costly to construct and limiting scalability to broader domains. The evaluation focuses primarily on mathematical reasoning, where final answers are automatically verifiable and intermediate reasoning can be assessed by human experts, leaving generalization to open-ended reasoning tasks uncertain. Furthermore, the work evaluates reward models only for best-of-N search rather than integrating

them directly into reinforcement learning, so it does not examine how process supervision behaves during policy optimization. Finally, the authors acknowledge potential test-set contamination in MATH and note that active learning strategies requiring repeated retraining remain unstable and require further investigation.

Extensions. This paper established process supervision as a practical alternative to outcome supervision and introduced the PRM800K benchmark, which became the foundation for much of the subsequent PRM literature. Later work has extended these ideas by developing scalable synthetic process supervision, improving reward model robustness against reward hacking, incorporating uncertainty-aware and calibrated reward estimation, and integrating PRMs directly into reinforcement learning pipelines. Future work could reduce reliance on costly human annotations through automated verification, improve process supervision under distributional shift, extend PRMs beyond mathematical reasoning to agentic tasks, and combine process supervision with robust optimization methods such as distributionally robust reinforcement learning or adversarial reward modeling.

Critical Comments. This paper is arguably the foundational work on process reward models, introducing both the PRM800K dataset and the empirical evidence that step-level supervision substantially outperforms outcome supervision. However, it studies PRMs only as verifiers for best-of-N search rather than as reinforcement learning reward functions. Consequently, many issues explored in later work—including reward hacking, calibration, robustness under policy optimization, and distributional shift—remain outside its scope. This distinction is important when positioning subsequent PRM research, which largely builds upon this work by addressing failures that emerge once PRMs are used for reinforcement learning rather than verification alone.

2.2. Towards Robust PRMs via Noise-Aware Learning

Link

TLDR: MC labels are policy-dependent and noisy (incorrect steps can have high reards if self correction succeeds). Propose Reflection aware label correction combined with iterative confidence-based denoising. Empirical study reports 27% abosulte F1 gain over noisy-supervision PRMs

Problem. Process Reward Models (PRMs) rely on accurate step-level supervision, but obtaining human annotations is prohibitively expensive. As a result, many recent approaches use Monte Carlo Estimation (MCE), which labels a reasoning step according to the probability that future rollouts from that step reach the correct final answer. The authors argue that this assumption is fundamentally flawed because the correctness of a reasoning step is an intrinsic property of the trajectory and should not depend on the capability of the policy generating future continuations. Consequently, MCE introduces systematic label noise through false positives (incorrect steps that are later corrected by the model) and false negatives (correct steps followed by later reasoning failures). The paper demonstrates that these policy-dependent errors persist regardless of sampling budget or model size, making noisy supervision a central obstacle to robust PRM training.

Approach. The authors propose a two-stage framework for robust PRM learning. First, during data annotation, they introduce a *reflection-aware label correction* mechanism that uses an LLM

judge to identify downstream self-correction or reflection. If a trajectory reaches the correct answer only because it later fixes an earlier mistake, that trajectory is no longer treated as evidence that the earlier reasoning step was correct, thereby reducing false-positive labels. Second, they introduce *Noise-Aware Iterative Training (NAIT)*, where an initially trained PRM repeatedly revises noisy labels using its own confidence estimates. Labels are only updated when the model strongly disagrees with the original annotation, allowing gradual refinement while preserving trustworthy supervision. Experiments show substantial improvements over standard MCE-trained PRMs, including up to a 27% absolute gain in step-level F1 and consistent improvements in Best-of-N reasoning performance across multiple mathematical reasoning benchmarks.

Limitations. Although the proposed framework substantially reduces annotation noise, it remains dependent on the quality of the LLM judge used during reflection-aware correction. If the judge fails to recognize subtle self-corrections or incorrectly flags valid reasoning, label quality may still degrade. Furthermore, the iterative relabeling procedure assumes that the model's confidence becomes increasingly reliable during training, an assumption that may not hold under severe distribution shift or highly noisy initialization. The evaluation is also restricted primarily to mathematical reasoning benchmarks and Best-of-N inference, leaving its effectiveness for reinforcement learning, general agentic reasoning tasks, and broader domains largely unexplored. Finally, the authors acknowledge that experiments were conducted only on relatively small Qwen2.5-Math models and were not extended to RL-based optimization due to computational limitations.

Extensions. This work opens several promising directions for robust process reward modeling. Rather than relying solely on reflection detection, future work could integrate uncertainty estimation, ensemble judges, or Bayesian confidence measures to identify noisy labels more reliably. The iterative label refinement framework could also be combined with adversarial training or distributionally robust optimization to improve robustness under distribution shift. An especially relevant extension is applying noise-aware supervision to process reward models used in reinforcement learning, where inaccurate rewards directly encourage reward hacking. Finally, extending the framework beyond mathematical reasoning to multi-step planning, coding, or embodied agents would help determine whether policy-dependent label noise is a general phenomenon in process supervision.

Critical Comments. This paper addresses one of the most fundamental robustness problems in PRMs by recognizing that the dominant source of supervision—Monte Carlo Estimation—is itself systematically biased. Unlike SCAN, which improves Monte Carlo annotation through confidence-based denoising, this work directly analyzes why MCE produces incorrect labels and proposes complementary mechanisms to correct both false positives and false negatives. The reflection-aware filtering stage is particularly compelling because it explicitly separates successful reasoning from successful recovery after an earlier mistake, a distinction ignored by traditional MCE. However, the framework remains largely heuristic and still depends on the reliability of external LLM judges and iterative self-correction. For research on robust PRMs, this paper provides a strong foundation for treating noisy supervision as a first-class robustness problem and suggests that future work should jointly model annotation uncertainty, process uncertainty, and adversarial robustness rather than relying on increasingly sophisticated Monte Carlo estimation alone.

2.3. CoLD: Counterfactually Guided Length Debiasing for PRMs in Math Reasoning

Link

TLDR: PRMs may rewards verbosity/length rather than correctness (a concrete reward-hacking shortcut). Propose counterfactual length debiasing through penalty, bias estimation, and joint-length invariant training. Empirical and causal motivation

Problem. Process Reward Models (PRMs) have become a central component of inference-time scaling by evaluating intermediate reasoning steps. However, the authors identify a previously overlooked failure mode: *length bias*. Existing PRMs consistently assign higher rewards to longer, more verbose reasoning steps even when those steps are semantically identical to shorter versions. Through controlled experiments using semantically equivalent rewrites and a causal analysis, the paper argues that step length acts as a confounding variable, causing PRMs to reward verbosity rather than logical correctness. This leads to unnecessarily long reasoning traces, unreliable reward estimates, and degraded downstream reinforcement learning performance.

Approach. The authors propose **CoLD (Counterfactually-Guided Length Debiasing)**, a framework motivated by causal inference and counterfactual reasoning. They model PRM predictions with a causal graph in which reasoning correctness and step length independently influence reward prediction, then explicitly remove the spurious length pathway. CoLD consists of three complementary components: (1) a deterministic length penalty that discourages excessive verbosity, (2) a learnable bias estimator trained to capture non-semantic length-related signals while being regularized with injected noise, and (3) joint optimization that encourages the PRM to become invariant to length while allowing the bias estimator to absorb verbosity-related effects. Extensive experiments on PRM800K, Math-Shepherd, MATH500, and GSM-Plus demonstrate improved step-selection accuracy, shorter reasoning trajectories, better downstream RL performance, and strong cross-domain generalization.

Limitations. Although CoLD substantially reduces verbosity bias, it addresses only one specific source of reward misspecification. Other PRM biases, including those arising from reasoning style, linguistic fluency, formatting, or annotation noise, remain unaddressed. The proposed causal graph assumes that length can be isolated from semantic correctness, an assumption that may weaken for naturally occurring reasoning where longer explanations are genuinely necessary. Moreover, nearly all experiments focus on mathematical reasoning, leaving uncertainty regarding effectiveness for broader reasoning domains or general-agent settings. Finally, CoLD introduces additional architectural complexity through the bias estimator and joint optimization, increasing training cost compared with standard PRMs.

Extensions. CoLD provides a promising foundation for broader robustness research in PRMs. The same counterfactual debiasing framework could be extended to remove additional spurious reward signals beyond length, such as stylistic preferences, formatting artifacts, or domain-specific shortcuts. Future work could integrate CoLD with robust PRM training methods, adversarial evaluation, calibration techniques, or uncertainty-aware reward estimation to produce reward models that remain reliable under distribution shift. It would also be valuable to evaluate CoLD within

larger agentic reasoning systems, reinforcement learning pipelines, and non-mathematical reasoning tasks where different forms of reward hacking may emerge.

Critical Comments. This paper makes an important contribution because it identifies a concrete and measurable robustness problem within PRMs rather than proposing another reward architecture. The causal framing provides a principled justification for why verbosity acts as a confounder, making CoLD more theoretically motivated than heuristic length penalties alone. For a literature review on robust PRMs, this work is particularly relevant because it demonstrates that PRM failures can arise from exploiting superficial correlations instead of reasoning quality. However, CoLD should be viewed as a targeted robustness solution rather than a general framework for robust reward modeling, since it focuses exclusively on length bias and does not address other sources of reward misalignment such as noisy supervision, adversarial reasoning, or distributional shift.

2.4. PathFinder-PRM: Error-Aware Hierarchical Supervision

Link

TLDR: PRMs need to know what kind of error occured, so the authors propose classifying math/consistency errors first and then combining signals into a correctness reward. Empirical study that improves PRMBench score with less data

Problem. Existing Process Reward Models (PRMs) supervise reasoning by assigning a single correctness score to each intermediate step, conflating two distinct objectives: identifying why a step is incorrect and estimating how useful that step is toward solving the problem. This entanglement limits the model's ability to recognize subtle reasoning failures such as mathematical mistakes, logical inconsistencies, redundancy, or deceptive reasoning, which have been highlighted by recent benchmarks such as PRMBench. Furthermore, current PRMs often require large quantities of annotated reasoning traces yet still struggle to generalize to nuanced error categories, motivating a more structured form of process supervision.

Approach. The authors propose **PathFinder-PRM**, an error-aware hierarchical Process Reward Model that explicitly separates error detection from reward estimation. Rather than directly predicting a step reward, the model first classifies each reasoning step according to two independent error dimensions: *mathematical errors* (e.g., incorrect calculations or invalid algebra) and *consistency errors* (e.g., contradictions with previous reasoning or the problem statement). These predicted error labels are then used as additional conditioning information to estimate the final process reward. To support this framework, the authors construct a new dataset containing approximately 400K reasoning trajectories by augmenting PRM800K and RLHFlow Mistral data with three-dimensional labels representing mathematical correctness, logical consistency, and overall step correctness. Extensive experiments on ProcessBench, PRMBench, and reward-guided mathematical reasoning demonstrate that this hierarchical supervision substantially improves fine-grained error detection while achieving state-of-the-art PRMBench performance despite using roughly one-third of the training data required by previous leading approaches.

Limitations. Although PathFinder-PRM improves robustness through hierarchical supervision, several limitations remain. The model currently distinguishes only two manually defined error categories, which may not capture richer reasoning failures such as missing assumptions, causal errors, or strategic planning mistakes. The approach also requires automatically generated error labels produced by another reasoning model, meaning annotation quality ultimately depends on the reliability of the labeling model. Experiments are restricted to mathematical reasoning benchmarks and 7B parameter models, leaving open questions regarding scalability to larger models, multimodal reasoning, coding, or agentic planning tasks. Finally, while separating error typing from reward estimation improves robustness, the method does not explicitly address adversarial reward hacking or optimization under reinforcement learning.

Extensions. The hierarchical framework naturally lends itself to richer forms of process supervision. Future work could expand the taxonomy of reasoning errors beyond mathematical and consistency violations to include planning failures, factual hallucinations, uncertainty, or missing prerequisite knowledge. Rather than relying on fixed manually designed categories, future models could learn latent error representations automatically through self-supervised or contrastive objectives. Integrating uncertainty estimation or distributionally robust optimization with hierarchical supervision could improve robustness under distribution shift and adversarial optimization. Finally, evaluating PathFinder-PRM within reinforcement learning or agentic reasoning systems would determine whether explicit error typing reduces reward hacking and leads to more reliable long-horizon reasoning.

Critical Comments. This paper introduces an intuitive yet effective modification to Process Reward Models by arguing that *understanding why a reasoning step is wrong should precede assigning it a reward*. Unlike previous work that primarily improves supervision quality through better annotations or architectural changes, PathFinder-PRM restructures the learning objective itself, explicitly decomposing error identification and reward estimation into sequential subtasks. The ablation studies strongly support this design, showing consistent improvements from both hierarchical supervision and separate error categories. For projects focused on robust PRMs, this work is particularly relevant because it reframes robustness as an interpretability problem: richer intermediate supervision leads to more reliable reward estimation. However, the method remains fundamentally discriminative and supervised, without explicitly considering adversarial optimization or formal robustness guarantees. Combining hierarchical error typing with uncertainty-aware PRMs, adversarial training, or distributionally robust reward learning represents a promising direction for developing Process Reward Models that are both interpretable and resistant to reward hacking.

2.5. Stop Summation: Min-Form Credit Assignment Is All Process Reward Model Needs for Reasoning

Link

TLDR: PRM reward hacking during Rl comes from summing future-step rewards, letting models exploit a few high-reward steps. Propose replacing cumulative rewards with minimum future reward so that one bad step limits trajectory value. Empirical study and claims training collapse under sum-form and improved reasoning with min-form.

Problem. Although process reward models (PRMs) provide dense, step-level supervision that has proven effective for test-time reasoning, they often fail when used for reinforcement fine-tuning. The authors argue that this failure is not primarily due to inaccurate reward models, but rather to the conventional summation-based credit assignment used in reinforcement learning. By defining the value of a reasoning step as the discounted sum of future process rewards, the policy is encouraged to maximize highly rewarded intermediate steps regardless of whether they contribute to solving the problem. This leads to severe reward hacking, including excessively long reasoning traces, degenerate outputs, and eventual training collapse.

Approach. The authors propose **PURE (Process sUpervised Reinforcement lEarning)**, which replaces the standard summation-form value function with a *min-form* credit assignment. Instead of valuing a reasoning trajectory by the cumulative future process rewards, PURE defines the value of a step using the minimum future process reward. Intuitively, this encourages the model to avoid weak reasoning steps rather than exploiting isolated high-reward ones, while also producing a bounded value function that is less sensitive to reward estimation errors. The framework is implemented within standard reinforcement learning algorithms without modifying the underlying PRM. Experiments on multiple Qwen2.5 models show that min-form credit assignment enables stable PRM-based reinforcement learning, achieves reasoning performance comparable to verifiable reward methods with substantially fewer optimization steps, and can be further improved by incorporating a small fraction of verifiable outcome rewards.

Limitations. While PURE substantially improves the stability of PRM-based reinforcement learning, it does not eliminate reward hacking entirely. The authors identify several remaining failure modes, including responses that endlessly "think" without producing solutions, degenerate outputs with extremely few reasoning steps, and collapse caused by repetitive training examples that receive incorrect positive process rewards. Furthermore, the method assumes access to reasonably accurate process reward models trained on high-quality annotations such as PRM800K, making its effectiveness dependent on PRM quality. The evaluation is also limited primarily to mathematical reasoning benchmarks, leaving its generalization to broader reasoning domains and agentic tasks largely unexplored.

Extensions. This paper shifts attention from improving reward model architectures toward improving how process rewards are incorporated into reinforcement learning. The proposed min-form credit assignment is complementary to advances in robust PRMs, uncertainty-aware reward modeling, and adversarial reward training, suggesting that both the reward function and the optimization objective should be designed jointly. Future work could extend PURE by developing adaptive or risk-sensitive credit assignment schemes, combining min-form objectives with calibrated uncertainty estimates from PRMs, or integrating distributionally robust optimization to make reinforcement learning less sensitive to reward misspecification and adversarial reasoning trajectories. These directions are particularly relevant for building robust process supervision methods that remain reliable under noisy or imperfect reward signals.

2.6. Adversarial Training for Process Reward Models

Link

TLDR: Instead of collecting more annotations, train in a GAN-esque manner (generate reasoning mistakes, detect afterwards) to introduce hard negative mining directly into PRM learning.

Problem. Most Process Reward Models (PRMs) are trained on static datasets containing either manually annotated reasoning steps or synthetic negative examples. While effective on in-distribution data, these approaches expose the reward model to only a fixed set of reasoning errors, limiting its ability to recognize increasingly subtle mistakes made by stronger language models. Furthermore, many synthetic data generation methods assume that a correct final answer implies correct intermediate reasoning, an assumption that breaks down when flawed reasoning accidentally reaches the correct solution. Consequently, existing PRMs often fail to generalize to novel reasoning errors and exhibit reduced robustness on out-of-distribution tasks.

Approach. The authors propose **Adversarially Trained Process Reward Models (APRM)**, which reformulates PRM training as a two-player adversarial game between a Generator and a Reward Model. The Generator learns, through reinforcement learning, to perturb correct reasoning steps into increasingly subtle but incorrect ones with the objective of fooling the PRM, while the PRM simultaneously learns to detect these adversarial errors. Unlike prior adversarial learning frameworks formulated as zero-sum games, APRM models the interaction as a general-sum game and introduces KL regularization, entropy regularization, and Optimistic Gradient Descent-Ascent (OGDA) to obtain theoretically stable learning dynamics with convergence guarantees toward a Nash equilibrium. The resulting adaptive curriculum continuously produces progressively harder negative examples without requiring additional manual annotation. Across multiple mathematical reasoning benchmarks, APRM consistently improves solver performance, achieving an average gain of 3.4 percentage points over the strongest existing PRM baseline and over 5 percentage points on out-of-distribution benchmarks.

Limitations. The primary limitation of APRM is its substantially higher training cost, as both a generator and reward model must be optimized simultaneously through reinforcement learning, even though inference-time cost remains unchanged. The method also depends on a handcrafted correctness oracle that combines symbolic reasoning, semantic similarity, structural validation, and heuristic rules to determine whether generated perturbations are actually incorrect. Errors or biases in this oracle may affect the quality of generated adversarial examples. In addition, most experiments focus on mathematical reasoning, leaving open questions regarding scalability to broader reasoning domains such as coding, legal reasoning, or multimodal tasks. Finally, while the adversarial framework improves robustness to novel reasoning errors, it does not explicitly address other PRM failure modes such as annotation noise, reward calibration, verbosity bias, or uncertainty estimation.

Extensions. The adversarial curriculum proposed by APRM provides several promising directions for robust PRM research. Future work could replace the rule-based correctness oracle with learned verifiers or theorem provers to generate higher-quality adversarial examples across diverse

domains. Multi-agent adversarial settings, where multiple generators target different classes of reasoning failures simultaneously, could further diversify negative samples. APRM could also be combined with recent work on uncertainty-aware PRMs, hierarchical reward models, or counterfactual debiasing methods to improve robustness against multiple sources of reward misspecification simultaneously. Finally, extending adversarial PRM training beyond mathematics to scientific reasoning, programming, and embodied agents would test whether adaptive curricula transfer across reasoning domains.

Critical Comments. This paper is one of the strongest robustness-oriented PRM papers because it directly addresses the core weakness of static reward model training: the inability to anticipate future reasoning errors. Rather than collecting larger datasets, APRM continuously adapts its training distribution by generating increasingly difficult counterexamples, making it conceptually similar to adversarial training in computer vision. The theoretical treatment distinguishing the problem as a general-sum game with convergence guarantees also strengthens the methodological contribution beyond empirical improvements. For research on robust PRMs, APRM demonstrates that adaptive data generation may be as important as reward model architecture itself. However, the framework primarily improves robustness to evolving reasoning errors rather than robustness to noisy labels, distribution shift, or reward hacking during reinforcement learning, making it complementary to recent work on calibration, debiasing, and robust reward estimation rather than a complete solution.

2.7. Preventing Process Reward Model Hacking When Training Large Language Models on Verifiable Rewards

Link

TLDR: Instead of improving the PRM, the reward formulation is modified using potential-based reward shaping and policy-invariant shaping so PRM rewards cannot alter optimal policy, supported both theoretically and empirically

Problem. Reinforcement Learning with Verifiable Rewards (RLVR) has emerged as an attractive alternative to preference-based alignment by optimizing language models using objective correctness signals available in domains such as mathematics and coding. Because these rewards are sparse, practitioners commonly supplement them with dense Process Reward Model (PRM) rewards that evaluate intermediate reasoning steps. The authors argue that this combination introduces a fundamental reward-shaping problem: if the dense PRM reward is not carefully designed, the policy may optimize the PRM instead of the verifiable objective. This leads to reward hacking, where models generate excessively long or plausible-looking chains of thought that maximize PRM rewards without improving final-answer correctness. The paper seeks to determine how dense PRM rewards can be incorporated into RLVR while guaranteeing that the optimal policy remains unchanged.

Approach. Rather than proposing a new process reward model, the authors reinterpret PRM rewards as a form of reward shaping and adapt two theoretically grounded reward-shaping methods—Generalized Reward Matching (GRM) and Action-Dependent Optimality-Preserving Shaping

(ADOPS)—to the RLVR setting. Both methods transform the dense PRM reward into an *optimality-preserving* shaping reward that provides informative intermediate feedback without changing which policy is ultimately optimal under the verifiable reward. The authors derive simplified formulations of both methods by exploiting the deterministic nature of chain-of-thought generation and prove that these transformations preserve optimality. Using the VersaPRM dataset, they demonstrate that standard PRM rewards introduce numerous false-positive and false-negative incentives for reward hacking, whereas both adapted GRM and ADOPS eliminate these incentives while maintaining useful dense supervision.

Limitations. As an extended abstract, the paper focuses primarily on theoretical analysis and proof-of-concept experiments rather than large-scale empirical validation. The evaluation is limited to the VersaPRM dataset and verifiable reasoning tasks, leaving open whether the proposed reward-shaping techniques generalize to broader process reward modeling settings such as open-ended reasoning, agentic planning, or tool use. Furthermore, the framework assumes the existence of reliable verifiable rewards, an assumption that does not hold for many alignment tasks where correctness cannot be automatically determined. Finally, the work modifies the reinforcement learning objective rather than improving the robustness of the process reward model itself, meaning vulnerabilities within the PRM remain if it is used outside an RLVR framework.

Extensions. The proposed optimality-preserving reward shaping framework offers a promising direction for integrating robust PRMs into reinforcement learning without encouraging reward hacking. Future work could investigate combining these shaping methods with uncertainty-aware or adversarially trained PRMs so that both the reward representation and optimization objective become more robust. Another natural extension is adapting the framework to settings where rewards are only partially verifiable or probabilistic, requiring uncertainty-aware versions of GRM or ADOPS. Finally, evaluating these methods in long-horizon agentic tasks, multi-step planning, or tool-using language models would help determine whether optimality-preserving shaping remains effective when reasoning becomes significantly more complex.

Critical Comments. This paper provides a unique perspective within the PRM robustness literature by approaching reward hacking from the reinforcement learning objective rather than the reward model itself. Unlike methods such as REFORM or Noise-Aware Learning, which attempt to improve the quality of reward supervision, this work assumes the PRM is imperfect and instead guarantees that its dense rewards cannot alter the optimal policy. The connection between process reward modeling and classical potential-based reward shaping is both elegant and theoretically grounded, making this one of the few papers to provide formal guarantees against PRM-induced reward hacking. However, its applicability is limited to RLVR settings with verifiable rewards, meaning many realistic reasoning tasks lacking objective correctness signals will still require fundamentally more robust process reward models in addition to optimality-preserving optimization.

2.8. Reward Under Attack: Analyzing the Robustness and Hackability of PRMs

Link

TLDR: Directly Studies PRM Hackability; finds that PRMs can reward fluent invalid reasoning and RL can drive near-perfect PRM scores while accuracy stays very low. Diagnostic framework: static perturbations, adversarial optimization, and RL induced hacking. An empirical robustness evaluation that results in a usable tool.

Problem. As Process Reward Models (PRMs) become increasingly integrated into reasoning systems and reinforcement learning pipelines, their robustness under optimization remains largely unexplored. Existing work has primarily evaluated PRMs based on offline accuracy or their ability to identify incorrect reasoning steps, but these metrics do not reveal whether a policy can exploit the reward model during optimization. The authors argue that this is analogous to classical reward hacking in reinforcement learning: if a PRM learns superficial features correlated with correct reasoning rather than reasoning itself, an optimizer may discover trajectories that receive high rewards despite producing incorrect solutions. Consequently, there is a need for systematic methods to evaluate PRMs under adversarial pressure rather than passive benchmark evaluation.

Approach. The authors propose a three-stage diagnostic framework that evaluates PRM robustness under progressively stronger optimization pressure. First, they perform *static perturbation analysis*, measuring whether PRMs remain invariant to semantics-preserving edits while penalizing logically corrupted reasoning. Second, they introduce *gradient-based adversarial token optimization*, treating the PRM as a differentiable objective and searching for token sequences that maximize reward on incorrect reasoning trajectories. Finally, they study *RL-induced reward hacking* by training language models using only PRM rewards and measuring whether increases in reward correspond to genuine improvements in reasoning accuracy. Across these evaluations, they show that current PRMs exhibit a "fluency–logic dissociation": they are highly robust to stylistic changes but inconsistently detect logical corruption. Furthermore, gradient-based attacks substantially inflate rewards on invalid reasoning, while reinforcement learning produces policies that achieve nearly perfect PRM rewards despite maintaining almost zero mathematical accuracy. The analysis further reveals distinct failure modes across models, with Skywork rewarding elaborate but incorrect reasoning and Qwen encouraging vacuous responses that avoid making substantive mathematical claims.

Limitations. While the paper provides one of the most comprehensive robustness evaluations for PRMs, it focuses exclusively on diagnosing vulnerabilities rather than proposing methods to improve robustness. The experiments are restricted to mathematical reasoning tasks and a small number of open-source PRMs, leaving open whether similar failure modes occur in broader reasoning domains such as coding, planning, or scientific reasoning. In addition, several adversarial experiments assume white-box access to model gradients, which may not always be available in practice. Although the reinforcement learning experiments demonstrate that optimization naturally discovers reward-hacking strategies, the work does not investigate mitigation techniques such as adversarial training, uncertainty-aware reward modeling, or robust reinforcement learning objectives.

Extensions. This paper provides an important evaluation framework for future research on robust PRMs by emphasizing that reward models should be evaluated under optimization rather than

only through offline accuracy. A natural extension would be to benchmark adversarially trained PRMs, uncertainty-calibrated PRMs, or ensemble reward models using the proposed diagnostic framework to determine whether they genuinely reduce exploitability. Another promising direction is incorporating distributionally robust optimization or adversarial trajectory generation during PRM training so that reward models become less sensitive to optimization-induced distribution shift. Finally, extending the benchmark beyond mathematical reasoning to agentic systems, tool use, and long-horizon planning would determine whether the observed reward-hacking behaviors generalize to more realistic deployment settings.

Critical Comments. This paper makes a valuable contribution by shifting the discussion of PRMs from predictive accuracy to optimization robustness. Rather than asking whether a PRM correctly evaluates reasoning traces, the authors ask whether those evaluations remain reliable once an optimizer actively searches for weaknesses. This distinction is highly relevant for reinforcement learning, where reward hacking arises precisely because policies optimize imperfections in the reward signal. Although the paper does not propose a more robust PRM, it establishes a rigorous evaluation methodology and demonstrates that many existing PRMs behave more like detectors of fluent reasoning than verifiers of logical correctness. As a result, this work provides a strong empirical motivation for developing robustness-aware PRM training methods and should serve as a standard benchmark for evaluating future robust process reward models.

3. Other Related Work

3.1. Towards Hierarchical Multi-Step Reward Models for Enhanced Reasoning in LLMs

Link

TLDR: Stepwise PRMs may mishandle multistep coherence and correction, which makes scores unreliable. The authors propose scoring individual steps and consecutive step blocks, and use hierarchical node compression for data augmentation. An empirical study was conducted and claims more stable best-of-N behavior and cross domain robustness

Problem. Conventional Process Reward Models (PRMs) evaluate each reasoning step independently, providing fine-grained supervision but often failing to capture coherence across multiple reasoning steps. As a result, PRMs are vulnerable to reward hacking, where models optimize local reward signals without producing genuinely correct reasoning. They also incorrectly penalize reasoning trajectories that temporarily contain an error but later recover through self-reflection or correction. In addition, existing PRMs rely on expensive human-annotated reasoning traces or computationally intensive Monte Carlo Tree Search (MCTS) for automatic annotation, limiting their scalability. The authors therefore aim to improve both the robustness of process reward modeling and the efficiency of generating high-quality supervision.

Approach. The authors propose the **Hierarchical Reward Model (HRM)**, which introduces hierarchical supervision by jointly learning from individual reasoning steps and short sequences of consecutive reasoning steps. During training, neighboring reasoning steps are merged to create coarse-grained supervision that captures both local correctness and global reasoning coherence.

Unlike standard PRMs, HRM can recognize situations where an incorrect intermediate step is subsequently corrected, rewarding the overall reasoning trajectory rather than prematurely penalizing isolated mistakes. Importantly, HRM retains standard step-wise inference, assigning rewards to individual reasoning steps at test time. To reduce the cost of automatic annotation, the paper also introduces **Hierarchical Node Compression (HNC)**, a lightweight data augmentation technique that merges consecutive nodes within MCTS-generated reasoning trees. HNC increases the diversity of automatically labeled reasoning trajectories while introducing only controlled label noise and negligible additional computational cost. Experiments on PRM800K demonstrate that HRM consistently produces more stable Best-of-N reasoning performance than conventional PRMs, while cross-domain evaluations on GSM8K and MATH500 show improved robustness and generalization.

Limitations. Although HRM improves robustness relative to conventional PRMs, its hierarchical supervision remains limited to merging pairs of consecutive reasoning steps. The authors acknowledge that extending the approach to longer reasoning segments would substantially complicate label construction due to the rapidly increasing number of possible label combinations. Furthermore, HRM is evaluated almost exclusively on mathematical reasoning datasets, making it unclear whether hierarchical supervision transfers effectively to broader reasoning domains such as coding, planning, or agentic workflows. The method also continues to depend on MCTS-generated supervision, whose computational cost remains the dominant bottleneck despite the efficiency gains introduced by HNC. Finally, while HRM mitigates some forms of reward hacking by evaluating reasoning coherence, it does not explicitly defend against adversarial optimization or reinforcement learning-based exploitation of reward models.

Extensions. The hierarchical supervision framework naturally suggests several promising research directions. Future work could extend HRM beyond fixed two-step reasoning segments by developing adaptive hierarchical structures capable of modeling longer reasoning dependencies without requiring manually specified labeling rules. Integrating HRM with uncertainty-aware reward models, adversarial training, or distributionally robust optimization could further improve robustness under distribution shift and optimization pressure. Another promising direction is replacing heuristic node merging with learned hierarchical representations that automatically identify meaningful reasoning units. Finally, evaluating HRM within reinforcement learning pipelines would help determine whether improved reasoning coherence also translates into greater resistance to reward hacking during policy optimization.

Critical Comments. This paper provides a novel perspective on PRM robustness by arguing that many reward-model failures arise from treating reasoning steps independently rather than as parts of a coherent reasoning process. Unlike work that focuses on improving annotation quality (e.g., SCAN or noise-aware supervision), HRM instead modifies the representation learned by the reward model itself, enabling it to recognize self-correction and long-range reasoning consistency. This hierarchical formulation directly addresses one of the weaknesses of conventional PRMs: rewarding isolated reasoning steps without considering their broader context. However, while HRM improves robustness against local reasoning inconsistencies, it does not provide formal guarantees against optimization-induced reward hacking or adversarial exploitation. Consequently, HRM represents a

complementary direction to existing robustness methods and could potentially be combined with uncertainty-aware reward modeling, adversarial training, or optimality-preserving reinforcement learning objectives to produce substantially more robust process reward models.

3.2. SCAN: Self-Denoising MC Annotation for Robust Process Reward Learning

Link

TLDR: Synthetic MC cupervision is scalable but noisy. Propose selective sampling + self denoising loss to reduce FP/FN. Empirical study with large F1 improvement and lower annotation cost

Problem. The paper addresses the difficulty of training Process Reward Models (PRMs) at scale without relying on expensive human step-level annotations. Monte Carlo annotation offers a cheaper alternative by estimating whether a reasoning step is correct based on whether sampled continuations reach the correct final answer. However, these labels are systematically noisy because they reflect the capabilities of the completer model rather than the true correctness of the current step. Weak completers may underestimate correct steps when they fail to finish the solution, while stronger models may overestimate incorrect steps by repairing earlier errors during rollout. Training directly on these labels can therefore cause PRMs to overfit annotation noise and perform poorly as the synthetic dataset grows.

Approach. The authors propose SCAN, which combines efficient Monte Carlo data generation with noise-robust PRM training. They first estimate the annotation model's self-confidence on each question and use it to identify more reliable training examples. High-confidence positive responses are retained without expensive stepwise rollout annotation, while Monte Carlo estimation is concentrated on informative negative responses. During training, the method applies soft labels to steps near the predicted first error, accounting for uncertainty in the precise error location. It also rescales step scores using the completer's question-level self-confidence to reduce bias caused by differences in annotation-model capability. Empirically, the resulting PRMs outperform standard Monte Carlo baselines, avoid the rapid overfitting observed under noisy supervision, and achieve competitive or superior performance to models trained on substantially larger human-annotated datasets.

Limitations. SCAN improves robustness to noisy synthetic labels, but its corrections depend on self-confidence being a meaningful proxy for annotation reliability. A model may be highly confident on familiar or biased problem types while remaining systematically wrong, so confidence-based filtering may preserve structured errors rather than remove them. The tolerance window around predicted error locations is also a manually selected hyperparameter and assumes that annotation mistakes occur locally. Moreover, the experiments focus on mathematical reasoning with automatically verifiable answers, where Monte Carlo completion is easier to evaluate than in open-ended domains. The paper evaluates PRMs primarily through best-of-N selection and first-error detection, rather than testing whether the trained reward models remain robust when optimized directly through reinforcement learning.

Extensions. This work is directly relevant to robust PRMs because it treats annotation noise as a central source of reward-model misspecification. A natural extension would replace heuristic confidence correction with calibrated uncertainty estimates, ensembles, or Bayesian models that distinguish epistemic uncertainty from task difficulty. Distributionally robust objectives could also be used to train against worst-case perturbations of Monte Carlo labels or shifts across completer models. Another direction would jointly learn the PRM and annotation policy, allocating additional rollouts only where uncertainty or disagreement is high. Finally, SCAN-trained PRMs should be evaluated under reinforcement learning to determine whether robustness to noisy labels also reduces downstream reward hacking, rather than only improving static error detection and best-of-N performance.

3.3. Know What You Don't Know: Uncertainty Calibration of PRMs

Link

TLDR: PRMs are often overconfident and overestimate probability of success. Quantile-Regression calibration and confidence bounds are used for adaptive test time compute, and results are empirical calibration and adaptive scaling

Problem. The rapid improvement of large language models has increased interest in process reward models (PRMs), which supervise intermediate reasoning steps instead of only evaluating final answers. Existing PRM benchmarks, however, primarily consist of mathematical reasoning tasks and therefore provide an incomplete picture of PRM capabilities. As reasoning models become increasingly general-purpose, it is unclear whether current benchmarks accurately measure reasoning quality across domains such as coding, scientific reasoning, or planning. The authors argue that the lack of diverse evaluation datasets limits our understanding of PRM robustness and obscures failure modes that emerge outside mathematics.

Approach. The authors introduce **PRMBench**, a comprehensive benchmark designed to evaluate process reward models across multiple reasoning domains. The benchmark contains human-annotated reasoning trajectories spanning mathematics, coding, science, and general reasoning, with fine-grained labels identifying both correct and incorrect intermediate reasoning steps. Rather than measuring only whether a PRM assigns high scores to correct solutions, PRMBench evaluates the model's ability to detect localized reasoning errors, distinguish subtle logical mistakes from correct reasoning, and generalize across heterogeneous task distributions. Extensive experiments compare both open-source and proprietary PRMs, revealing that strong performance on mathematical reasoning does not necessarily translate to other reasoning domains. The benchmark therefore exposes significant gaps in the generalization ability of current PRMs despite their impressive performance on existing math-focused evaluations.

Limitations. Although PRMBench substantially broadens PRM evaluation, it remains an evaluation benchmark rather than a training methodology for improving robustness. Performance is also influenced by annotation quality, as accurately labeling intermediate reasoning steps across diverse domains is inherently difficult and expensive. Furthermore, while the benchmark includes multiple reasoning categories, it still evaluates relatively static reasoning traces and does not fully capture

agentic behaviors involving long-horizon planning, tool use, or adaptive interaction with external environments. Consequently, important robustness challenges such as reward hacking, adversarial reasoning, and distributional shift remain only partially addressed.

Extensions. PRMBench provides a valuable foundation for developing robustness-oriented process reward models. Future work could incorporate adversarially generated reasoning traces, distribution-shifted reasoning tasks, or intentionally misleading intermediate steps to evaluate robustness under optimization pressure rather than standard reasoning accuracy alone. The benchmark could also be extended to interactive agent settings where reasoning evolves dynamically through tool use and environmental feedback. Combining PRMBench with adversarial training, uncertainty estimation, or distributionally robust optimization would enable systematic evaluation of whether robust PRMs genuinely resist reward hacking while maintaining strong reasoning performance across diverse domains.

Critical Comments. This paper is highly relevant because robust PRM research requires reliable evaluation before meaningful improvements can be measured. Existing benchmarks often overestimate PRM capabilities by focusing almost exclusively on mathematics, whereas PRMBench demonstrates that current models generalize poorly across broader reasoning tasks. Although the paper does not directly propose robustness methods, it identifies a critical evaluation gap that future robustness techniques—including adversarial training, robust optimization, and reward-hacking defenses—can use as a standardized benchmark. For projects investigating robustness in process reward models, PRMBench represents an important evaluation framework against which new robustness methods should be validated.

3.4. SOCRATIC-PRMBENCH: Benchmarking Process Reward Models with Systematic Reasoning Patterns

Link

TLDR: Existing PRM benchmarks miss systematic reasoning-pattern failures. The authors propose to benchmark errors across decomposition and then combine signals into a correctness reward, results improve PRMBench score with less data

Problem. Existing Process Reward Model (PRM) benchmarks primarily evaluate whether models can identify incorrect reasoning steps, but they largely ignore *how* reasoning is performed. In practice, large language models employ diverse reasoning strategies such as decomposition, transformation, verification, and integration, each of which introduces distinct failure modes. Current benchmarks typically treat all reasoning errors uniformly, making it difficult to determine whether PRMs genuinely understand reasoning or merely recognize common error patterns. As a result, PRMs may successfully detect downstream mistakes while failing to identify the earlier reasoning errors that caused them, leading to delayed error detection and increasing the risk of error propagation and reward hacking during reinforcement learning.

Approach. The authors introduce **SOCRATIC-PRMBENCH**, the first benchmark designed to systematically evaluate PRMs across six fundamental reasoning patterns inspired by Socratic

logic: *Transformation*, *Decomposition*, *Regather*, *Deduction*, *Verification*, and *Integration*. Rather than simply labeling reasoning steps as correct or incorrect, the benchmark categorizes failures into 20 fine-grained reasoning error types and contains 2,995 reasoning trajectories with controlled error injections. A specialized Socratic reasoning model first generates correct reasoning traces, after which GPT-4o injects targeted reasoning errors corresponding to each category. Extensive filtering using rule-based checks, Gemini-2.5-Pro verification, and human annotation ensures dataset quality. The benchmark evaluates both dedicated PRMs and modern LLMs acting as critic models using the PRM-Score metric, revealing not only overall performance but also weaknesses across individual reasoning patterns and error categories.

Limitations. Although SOCRATIC-PRMBENCH provides a much richer evaluation framework than previous benchmarks, it remains an evaluation benchmark rather than a method for improving PRMs. Consequently, it diagnoses weaknesses without proposing new training algorithms to address them. Furthermore, the benchmark focuses almost exclusively on mathematical reasoning problems with objectively verifiable answers, limiting conclusions about domains such as medicine, law, or scientific reasoning where intermediate reasoning is often subjective. The benchmark also relies heavily on LLM-generated reasoning traces and synthetic error injection, which, despite extensive filtering, may not perfectly reflect naturally occurring reasoning failures generated by deployed models. Finally, while the benchmark identifies deficiencies related to reward hacking, latency, and reward bias, it does not evaluate PRMs under reinforcement learning where these issues may become substantially more severe.

Extensions. SOCRATIC-PRMBENCH establishes a valuable diagnostic framework for future PRM research. The benchmark naturally enables development of reasoning-pattern-aware PRMs that explicitly model different reasoning operations instead of treating all reasoning steps identically. Future work could use the benchmark for curriculum learning, adversarial training, uncertainty calibration, or distributionally robust optimization targeted toward underrepresented reasoning patterns. Another promising direction is extending the benchmark beyond mathematical reasoning into domains such as coding agents, medical diagnosis, legal reasoning, or multi-agent planning, where reasoning patterns are more diverse and errors are less objectively verifiable. Finally, the benchmark could be incorporated directly into reinforcement learning pipelines to evaluate whether improvements in reasoning-pattern robustness translate into reduced reward hacking and more reliable policy optimization.

Critical Comments. This paper makes an important conceptual contribution by arguing that robustness should be evaluated with respect to *reasoning patterns*, not merely reasoning correctness. Rather than introducing another dataset of correct and incorrect reasoning traces, it identifies systematic blind spots in existing PRMs, including poor performance on decomposition and transformation reasoning, delayed detection of early reasoning errors, and strong reward bias toward either positive or negative rewards. These findings help explain why PRMs remain vulnerable to reward hacking: if a reward model fails to recognize the reasoning operation being performed, it may incorrectly reward trajectories that appear locally plausible while containing fundamental structural errors. Although the work does not improve PRMs directly, it provides one of the

strongest evaluation frameworks currently available for measuring robustness and should become a standard benchmark for future research on robust process reward modeling.

3.5. PRISM: Probabilistic Reward Model with Inherent Structural Modeling

Link

TLDR: Scalar reward models are fundamentally brittle, so GMMs and uncertainty estimates are introduced

Problem. Conventional reward models compress diverse human judgments into a single scalar score, implicitly assuming the existence of an "average annotator." This simplification conflates two fundamentally different sources of disagreement: *subjective preference*, where multiple opinions are equally valid (e.g., humor versus safety), and *epistemic uncertainty*, where disagreement stems from ambiguity or insufficient information. The authors argue that this structural mismatch produces brittle reward models that are easily exploited during reinforcement learning, leading to reward hacking, poor calibration, and weak generalization under distribution shift. Existing approaches based on scalar Bradley–Terry reward modeling therefore fail to faithfully represent heterogeneous human preferences and provide unreliable optimization signals.

Approach. The authors propose **PRISM** (Probabilistic Reward Model with Inherent Structural Modeling), which models rewards as a conditional *Mixture of Gaussians* rather than a single scalar. Each Gaussian expert learns a distinct preference dimension, while its variance explicitly represents epistemic uncertainty. Training proceeds in two stages: first, uncertainty-aware experts are learned from large-scale pairwise preference data, where uncertain experts automatically suppress their own gradients instead of averaging conflicting supervision. Second, the experts are frozen and an input-conditioned router is trained to dynamically combine them based on the prompt context. This architecture enables PRISM to distinguish subjective disagreement from genuine uncertainty while remaining compatible with standard pairwise preference datasets. Experiments demonstrate improved accuracy on multiple reward-model benchmarks, better out-of-distribution generalization, and substantially more stable reinforcement learning compared to scalar reward models. Most notably, PRISM consistently mitigates reward hacking during Rubric-based Reinforcement Learning by providing a richer optimization landscape through uncertainty-aware reward distributions rather than single scalar rewards.

Limitations. Although PRISM models rewards probabilistically, downstream reinforcement learning algorithms still consume rewards largely through scalarized objectives, preventing the policy from fully exploiting the learned reward distribution. Consequently, much of the expressive power of the mixture representation is lost during optimization. Furthermore, the method introduces additional architectural complexity through multiple experts and routing networks, increasing training cost relative to conventional reward models. Evaluation is also limited by the scarcity of publicly available datasets containing diverse human preferences across multiple domains, making it difficult to fully assess robustness under realistic alignment settings. Finally, while PRISM substantially reduces reward hacking, it does not provide theoretical guarantees against adversarial optimization and remains dependent on the quality of the underlying preference data.

Extensions. PRISM naturally motivates several directions for robustness research. The most immediate extension is to design *distribution-aware reinforcement learning* algorithms that optimize directly over reward distributions instead of collapsing them into scalar objectives. Such methods could separately optimize expected reward, disagreement among experts, and predictive uncertainty, thereby preserving the information captured by the probabilistic reward model throughout policy optimization. Future work could also integrate uncertainty-aware exploration, adversarial robustness, or distributionally robust optimization to further reduce reward exploitation under distribution shift. Finally, extending PRISM to process reward models offers a promising avenue for modeling uncertainty across reasoning trajectories, enabling more reliable credit assignment and greater robustness against process-level reward hacking in agentic reasoning systems.

Critical Comments. This paper is highly relevant to robustness in reward modeling because it identifies the representation of rewards—not merely the optimization objective—as a major source of reward hacking. Unlike prior work that modifies reinforcement learning objectives (e.g., PURE), PRISM instead enriches the reward representation itself by explicitly modeling preference diversity and uncertainty. The uncertainty-gated mixture formulation provides a principled mechanism for reducing overconfident supervision and demonstrates measurable improvements in robustness during downstream RL. However, the paper also highlights an important open problem: current reinforcement learning algorithms still optimize scalar rewards, leaving much of the probabilistic information unused. For robustness-oriented process reward models, this suggests that future work should jointly develop probabilistic PRMs alongside distribution-aware optimization algorithms, rather than improving either component independently.

3.6. Teach a Reward Model to Correct Itself: Reward Guided Adversarial Failure Discovery for Robust Reward Modeling

Link

TLDR: Model generates its own adversarial failures and retrains on itself, self-play for reward models show 45% robustness improvement without needing manually designed attacks

Problem. Reward models are widely used to align large language models through reinforcement learning, response ranking, and response filtering, yet they frequently fail under distributional shift and adversarial perturbations. Existing approaches for identifying these failure modes typically require prior knowledge about the data distribution or manually designed adversarial attributes, making them difficult to deploy in realistic settings where unknown vulnerabilities are most likely to arise. The authors argue that robust reward modeling requires automatically discovering a reward model's blind spots before they are exploited during downstream optimization.

Approach. The authors propose **REFORM** (Reward-guided Adversarial Failure Discovery for Robust Reward Modeling), a self-improving framework that enables reward models to identify and correct their own weaknesses. Rather than relying on external adversarial datasets, REFORM uses the reward model itself to guide controlled decoding from an aligned language model, generating responses that are intentionally mis-scored by the reward model. These adversarial examples expose failure modes where preferred responses receive low rewards or dispreferred responses receive

high rewards. The discovered failures are then added back into the training data, allowing the reward model to retrain on precisely the examples that reveal its weaknesses. Experiments on the Anthropic Helpful-Harmless and PKU Beavertails datasets demonstrate that REFORM improves robustness against adversarial inputs while maintaining or slightly improving standard reward-model performance. Importantly, downstream reinforcement learning policies trained using the improved reward model also exhibit stronger alignment, suggesting that repairing reward-model vulnerabilities benefits the entire RLHF pipeline.

Limitations. Although REFORM provides an elegant mechanism for self-improving reward models, it remains dependent on the quality of the underlying aligned language model used for controlled decoding. If the generator fails to discover sufficiently diverse or difficult adversarial examples, important reward-model vulnerabilities may remain undetected. Furthermore, the paper focuses primarily on preference reward models used for alignment rather than process reward models that supervise intermediate reasoning steps. Consequently, it remains unclear whether the same adversarial failure discovery framework would expose reasoning-specific weaknesses such as incorrect logical verification or reward hacking in PRMs. The work also evaluates robustness primarily on preference benchmarks, leaving broader reasoning and agentic settings unexplored.

Extensions. The core idea of allowing reward models to generate their own adversarial training data naturally extends to process reward models. Rather than generating adversarial final responses, future work could generate adversarial reasoning trajectories that maximize PRM scores despite containing flawed intermediate reasoning, creating an automated curriculum for robust PRM training. REFORM could also be combined with adversarial optimization methods, uncertainty-aware reward models, or distributionally robust optimization so that discovered failure modes are incorporated into a broader robustness objective. Finally, repeatedly alternating between adversarial failure discovery and reward-model retraining suggests a continual self-improvement framework capable of adapting as increasingly capable reasoning models emerge.

Critical Comments. This paper is particularly relevant because it moves beyond evaluating reward-model robustness toward actively improving it. Unlike prior work that primarily diagnoses reward hacking, REFORM introduces a practical training loop that identifies and patches vulnerabilities before they can be exploited. Although the paper focuses on preference reward models rather than PRMs, the underlying philosophy aligns closely with robust process supervision: robustness should emerge from exposing reward models to their own failure cases rather than relying solely on static human annotations. Extending this framework to process reward models represents a promising direction for developing reasoning supervisors that are substantially more resistant to optimization-induced reward hacking.

Part II
Notes

basics of RL, how it works, and why rewards hackng occurs, is an issue, set up RL experiments. LLMs for math reasoning, benchmarking LLMs, MDPs

4. Reinforcement Learning Overview

RL is a sequential decision-making framework in which agents learn to perform actions in an environment with the goal of maximizing rewards. An RL algorithm might control the moves (*action*) of a character (*agent*) in a video game (*environment*) aiming to maximize the score (*reward*). In Finance, this could be a virtual trader who buys/sells assets on a financial exchange to maximize profit RL is a framework that doesn't necessarily need deep learning but modern systems often use deep networks by encoding the environment and mapping to the next action

Challenges of RL

Consider playing a game of chess. The reward set is $[-1, 0, +1]$ at the end of a game, representing wins, losses, and draws.

- The reward is *sparse* since we must play an entire game to recieve feedback.
- The reward is temporally offset from the action that caused it (an advantage might be gained 30 moves before victory) so we must associate this reward to its critical action. This is known as the *temporal credit assignment problem*
- The environment is stochastic since the opponent doesn't necessarily use the same moves.
- The agent must balance exploring the environment (trying new opening moves) and exploiting what it already knows. This is called *explroation-exploitation trade-off*.

5. Markov Decision Processes (MDPs), Returns, and Policies

RL learns a *policy* that maximizes *expected return* in a MDP.

A Markov process (MP) consists of a set of states and transition probabilities $\mathbb{P}(s_{t+1}|s_t)$ that define the probability s_t moves to s_{t+1}. Being Markovian means that the current state only depends on the previous state.

A Markov reward process (MRP) extends a MP to include a distribution $\mathbb{P}(r_{t+1}|s_t)$ over the possible rewards recieved at the next time step, producing a sequence of $s_1, r_2, s_2, r_3, s_3, r_4, ...$ and includes a discount factor $\gamma \in (0, 1]$ that is used to compute return G_t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

The return is the sum of the cumulative discounted future rewards, and $\gamma \leq 1$ makes rewards that are closer in time more valuable than rewards further away.

A Markov Decision Process (MDP) adds a set of possible actions at each time step, so the reward depends on action and state:

$$\mathbb{P}(s_{t+1}|s_t, a_t) \quad \mathbb{P}(r_{t+1}|s_t, a_t)$$

which produce tuples $(s_1, a_1, r_2), (s_2, a_2, r_3), (s_3, a_3, r_4), ...$

Partially Observable MPs (POMDP)

In this case, the agent does not have access to the entire space (think a chess piece only being able to move based on everything in a two tile radius). The agent recieves an observation $o_t \sim \mathbb{P}(o_t|s_t)$, generating $s_1, o_1, a_1, r_2, s_2, o_2, a_2, r_3, o_3, a_3, s_3, r_4, ...$

The rules that determine the agent's action for each state are known as a policy, which may be:

- stochastic: policy defines a distribution over actions for each state
- deterministic: agent takes the same action in a given state and zero otherwise
- stationary: depends only on the current state
- nonstationary: depends only on current time step

6. Expected Return

We want to choose a policy that maximizes expected return, so to do so we assign a value to each state and stata-action pair. The return G_t depends on the state s_t and policy $\pi(a|s)$. From this state, the agent will pass through a sequence of states, taking actions and recieving rewards. This sequence differs every time th eagent starts in the same place since the policy, state transitions, and rewards are all stochastic. We characterize how good a state is by considering the expected return:

$$v(s_t|\pi) = \mathbb{E}[G_t|s_t, \pi]$$

which informally tells us the long term reward we can expect. Similarity the state-action value function can tell us the expected reward if we start in a state, take an action, and follow the policy. This is how RL connects future rewards to current actions (thereby resolving the temporal credit assignment problem)

$$q(s_t, a_t|\pi) = \mathbb{E}[G_t|s_t, a_t, \pi]$$

6.1. Optimal Policy

We want a policy that maximizes expected return. For MDPs (but not POMDPs) there is always a deterministic stationary policy that maximisez. If we know this optimal policy, then the optimal state-value function $v^*[s_t]$ and stata-action value function $q^*[s_t, a_t]$ are:

$$v^*[s_t] = \max_\pi[\mathbb{E}[G_t|s_t, \pi]] \text{ and } q^*[s_t, a_t] = \max_\pi[\mathbb{E}[G_t|s_t, a_t, \pi]]$$

so if we knew the optimal action values, then we can derive the optimal policy by choosing the action with the highest value. Some RL algorithms are based on alternately estimating the action

values and policy

6.2. Bellman Equations

We may not know the state values or action values for any policy, but we know they must be consistent with one another. The state value can be found by taking a weighted sum of the action values with weights dependent on the PDF of the policy. Similarily, the value of an action is the immediate reward generated by taking the action plus the value $v[s_{t+1}]$ of being in the subsequent state s_{t+1} discouted by γ. This isnt deterministic, so we weight the values according to the transition probabilities:

$$q[s_t, a_t] = r[s_t, a_t] + \gamma \sum_{s_{t+1}} \mathbb{P}(s_t + 1 | s_t, a_t) v[s_{t+1}]$$

7. Tabular Reinforcement Learning

Tabilar RL algorithms (ones that don't rely on function approximation) are divided into *model-based* and *model-free mmethods*. Model-based methods use the MDP structure and find the best policy from the transition matrix and reward structure. If these are known, they can be tackled by dynamic programming, else they are estimated from observed trajectories.

Model free methods assume the transition matrix and reward structure of the underlying MDP are unknown, falling into two families:

- Value Estimation: estimate the optimal state-action value function and the assign policy according to the action in each state with the greatest value
- Policy Estimation: estimate the optimal policy using gradient descent without intermediate steps

Within each family, Monte Carlo is used to simulate and TD (temporal difference) updates the policy as the agent traverses the MDP

7.1. Dynamic Programming

Imagine a grid with a penguin looking for a fish, but the grid has holes as well. A DP algorithm follows:

1. Initialization: The state values are initialized at zero, and policy is chosen randomly.
2. Policy Evaluation: The state values are updated to be consistent with their neighbors.
3. Policy Improvement: The policy is updated to mobe the agent to states with the highest value
4. After several iterations, the algorithm converges to the optimal policy

Policy Evaluation We sweep through states s_t an update their values using

$$v[s_t] \leftarrow \sum_{a_t} \pi[a_t | s_t] (r[s_t, a_t] + \gamma \sum_{s_{t+1}} \mathbb{P}(s_{t+1} | s_t, a_t) v[s_{t+1}])$$

Each value is consisgtent with the successor state using the Bellman equation for state values, also known as *bootstrapping*.

Policy Improvement To update the policy, we greedily choose the action that maximizes the value for each state: